

AI Agent: 자연어 이해와 통제 실행을 분리하기

업무용 Agent의 핵심은 답변 생성이 아니라, 의도를 이해한 뒤 허용된 실행만 안전하게 수행하는 것이다.

요약

업무용 Agent를 설계하며 Rasa, Agent SDK, PlanVM, DBHub의 역할을 분리했다. Rasa는 정형 이해 힌트 계층, Agent SDK는 후보 조정과 설명 계층, PlanVM은 통제된 실행 계층, DBHub는 운영 DB 접근 통제 계층으로 보았다.

핵심 판단은 자율형 Agent를 그대로 업무 실행에 연결하지 않는 것이다. 자연어는 모호하고, 업무 실행은 되돌리기 어렵다. 따라서 의도·slot·맥락 판단과 실제 실행을 분리하고, 실행은 승인된 plan과 도구 계약 안에서만 수행해야 한다.

문제 정의

사용자는 “엑셀 분석해줘”, “이번 달 미처리 건 확인해줘”, “고객사 설정 바꿔줘”처럼 자연어로 요청한다. 같은 문장 안에는 조회, 분석, 변경, 권한, 승인, 파일 처리, DB 접근이 섞일 수 있다. LLM이 곧바로 tool을 호출하면 다음 위험이 생긴다.

- 의도 분류가 흔들리면 잘못된 업무가 실행된다.
- slot 누락 상태에서 실행하면 잘못된 tenant나 기간을 사용할 수 있다.
- fallback chain이 길어지면 책임 소재가 불명확해진다.
- DB 접근과 외부 호출이 audit 없이 발생할 수 있다.
- 조직별 용어와 alias가 반영되지 않으면 잘못된 해석이 발생한다.

책임 경계

계층	책임
Rasa	intent, entity, slot, form, rule 기반 정형 이해 힌트를 제공한다.
Agent SDK	LLM 기반 후보 조정, 설명, 대화 맥락 정리를 수행한다.
PlanVM	승인된 업무 실행 plan만 호출한다.
DBHub	운영 DB 조회·변경을 권한, masking, audit 기준으로 제한한다.

계층	책임
Dictionary	조직 용어, 직책, 제품명, 거래처명, alias를 관리한다.
Operator Gate	위험한 실행을 승인 흐름으로 전환한다.

핵심 설계 결정

Rasa를 전체 Agent가 아니라 이해 힌트 계층으로 사용했다

Rasa는 intent, slot, rule, form 기반의 정형 이해에 강점이 있다. 그러나 대화 전체를 Rasa에 맡기면 catalog가 커질수록 학습·운영·fallback 관리가 복잡해질 수 있다. 따라서 Rasa는 자연어 이해의 선행 힌트를 제공하고, 대화 조정은 Agent SDK가 담당하는 구조로 분리했다.

이 구조에서 Rasa의 결과는 최종 실행 명령이 아니라 후보 신호다. Agent는 Rasa 결과, 사용자 맥락, 조직 용어사전, tool catalog, 권한 정보를 함께 보고 실행 후보를 좁힌다.

fallback chain을 제한했다

업무용 Agent에서 fallback은 안전장치이지만, fallback chain이 길어지면 시스템은 설명 불가능해진다. “Rasa가 실패하면 LLM, LLM이 실패하면 다른 tool, 다시 retry” 같은 흐름은 장애 분석과 감사에 취약하다.

따라서 fallback은 명시적인 상태 전환으로 제한해야 한다.

```
understand
-> clarify
-> validate slots
-> select approved plan
-> request approval if needed
-> execute
-> audit
```

불확실하면 실행하지 않고 질문하거나 승인 단계로 올리는 것이 업무용 Agent에는 더 안전하다.

조직 용어사전을 핵심 자산으로 봤다

업무용 자연어 이해에서 가장 위험한 것은 모델 성능만이 아니다. 조직마다 제품명, 부서명, 직책, 거래처명, 약어, 오타, 별칭이 다르다. 이 정보를 catalog와 alias registry로 관리하지 않으면 Agent는 같은 단어를 다른 의미로 해석할 수 있다.

따라서 도메인 catalog와 조직 용어사전은 선택 기능이 아니라 운영 필수 자산이다. 용어 변경, alias 충돌, 승인, publish, rollback까지 관리되어야 한다.

버린 선택과 이유

LLM에게 업무 실행을 전부 맡기는 방식은 채택하지 않았다. 자연어 이해에는 강점이 있지만, 업무 시스템에서는 권한, 승인, 감사, rollback, 실행 계약이 더 중요하다. “그럴듯한 답변”과 “허용된 업무 실행”은 다른 문제다.

Rasa를 전체 대화 엔진으로 고정하는 방식도 제한했다. intent, slot, rule은 정형 업무 이해에 강하지만, 도메인이 많아지고 조직별 용어가 달라지면 모든 흐름을 rule로 유지하기 어렵다. 그래서 Rasa는 이해 힌트 계층으로 두고, 실행은 별도 계약 계층으로 넘기는 구조를 검토했다.

fallback chain을 길게 두는 방식도 피했다. 이해가 안 되면 계속 다른 모델과 룰로 넘기는 구조는 겉으로는 유연하지만, 운영에서는 원인 파악이 어려워진다. 업무용 Agent는 모르는 것을 인정하고, 확인 질문이나 operator review로 전환할 수 있어야 한다.

업무용 Agent에서 남겨야 하는 기록

Agent 실행은 자연어 요청 하나로 끝나면 안 된다. 다음 기록이 남아야 업무 시스템에 연결할 수 있다.

기록	목적
intent/slot 후보	Agent가 무엇을 이해했다고 판단했는지 검증한다.
조직 용어 매핑	사용자 표현이 어떤 업무 용어로 해석되었는지 확인한다.
선택된 tool/plan	실제 실행 경로를 audit한다.
confidence와 review 여부	자동 실행과 사람 검토의 경계를 남긴다.
실행 결과 trace	실패 시 재실행, 보정, rollback 판단에 사용한다.

구현 구조

User Message

- > Rasa NLU / Rule / Slot Hint
- > Agent SDK orchestration
- > Dictionary / Alias resolution
- > Permission / Risk check
- > PlanVM approved plan
- > DBHub / HTTP / Task / File tool
- > Audit / Result explanation

Agent가 직접 DB나 외부 시스템을 호출하지 않는다. 실행은 PlanVM과 DBHub 같은 통제 계층을 통과한다. 위험도가 높은 작업은 operator review gate를 거친다.

시스템 설계자의 그림

업무용 Agent에서 가장 중요한 경계는 자연어 이해와 실행 사이에 있다. 이해 계층은 후보를 만들 수 있지만, 실행 계층은 허용된 plan과 tool 계약만 수행해야 한다. 모호하면 질문하고, 위험하면 검토로 올리며, 지원하지 않는 요청은 차단해야 한다.

운영 검증 {#운영-검증 }

검증 기준은 “답을 잘하는가”보다 “잘못 실행하지 않는가”였다. 특히 다음 항목을 중점으로 봤다.

- slot이 부족할 때 실행하지 않고 질문하는가.
- 도메인 catalog가 커져도 의도 분류가 흔들리지 않는가.
- 조직 alias 충돌을 감지하는가.
- 위험 작업은 승인 단계로 전환되는가.
- 실행 결과와 사용된 plan, parameter, actor가 audit으로 남는가.

역할과 책임

업무용 Agent의 책임 경계, Rasa와 Agent SDK 조합 방식, fallback 제한, PlanVM 실행 연동, DBHub 통제, 도메인 catalog, 조직 용어사전과 alias registry의 운영 기준을 설계했다.

설계 결론

업무용 Agent는 채팅 모델이 아니라 실행 시스템이다. 자연어 이해는 중요하지만, 최종 가치는 안전한 실행, 권한 통제, 감사, 재현 가능성에서 나온다. Agent가 할 일은 자유롭게 무엇이든 하는 것이 아니라, 조직이 허용한

업무 실행 경로를 정확히 선택하고 설명하는 것이다.